INSTITUCION EDUCATIVA UNIVERSITARIA
PASCUAL BRAVO

BASE DE DATOS I

Tarea. Comparar la implementación de un diccionario de datos físico en diferentes SGBD

ORALIA CORTES GRAJALES

JEISSON JAVIER OYOLA TIQUE

25/10/2026

1. RESUMEN EJECUTIVO El presente informe documenta el desarrollo, modelado e implementación del diccionario de datos físico para el núcleo de una plataforma de red social estudiantil universitaria. A través de este documento se aborda el proceso desde el diseño conceptual y lógico hasta la definición física de las tablas de la base de datos en el gestor relacional PostgreSQL. Se analiza la justificación técnica de la selección de tipos de datos, el establecimiento de restricciones de integridad referencial y de dominio, y la estructura requerida para garantizar un rendimiento óptimo, escalabilidad y coherencia en la persistencia de la información.

2. INTRODUCCIÓN Y JUSTIFICACIÓN DEL PROYECTO En el contexto actual del desarrollo de software, la arquitectura de persistencia de datos constituye el pilar fundamental para garantizar la disponibilidad, la integridad y el rendimiento de las aplicaciones web y móviles. Para el desarrollo de una red social estudiantil enfocada en la comunidad universitaria, el sistema debe ser capaz de gestionar de forma eficiente la información de los usuarios, sus interacciones y la trazabilidad de sus actividades. El propósito de la Tarea 4 consiste en estructurar formalmente el diccionario de datos físico y comparar su comportamiento e implementación a través de distintos Sistemas de Gestión de Bases de Datos (SGBD) Relacionales. Este informe expone detalladamente el modelo implementado en PostgreSQL mediante la herramienta pgAdmin 4, justificando cada decisión técnica tomada respecto a los tipos de datos (como la selección entre cadenas de longitud fija vs. variable, números enteros e intervalos temporales) y las reglas de negocio traducidas en restricciones del sistema.

3. ARQUITECTURA Y DISEÑO DEL MODELO DE DATOS El modelo de datos se diseñó bajo los principios de normalización relacional (Tercera Forma Normal - 3FN) con el objetivo de eliminar la redundancia de información y prevenir anomalías durante las operaciones de inserción, actualización y borrado.

3.1. Caso de Estudio: Red Social Estudiantil La entidad principal evaluada en esta entrega corresponde al módulo de Usuarios / Estudiantes (id_usuario), la cual actúa como entidad madre para el ecosistema de la red social (publicaciones, comentarios, conexiones y autenticación). 4. DICCIONARIO DE DATOS FÍSICO A continuación se

presenta la especificación técnica del diccionario de datos definido para la entidad usuarios:

Nombre del Campo	Tipo de Dato	Tamaño / Formato	Restricciones de Integridad	Descripción Técnica
id_usuario	Entero (INT / SERIAL)	8 Bytes	Clave Primaria (PRIMARY KEY), No Nulo (NOT NULL), Único (UNIQUE), Auto-incremental	Identificador único numérico asignado secuencialmente por el motor de base de datos para cada usuario.
nombre_usuario	Cadena (VARCHAR)	100 caracteres	No Nulo (NOT NULL)	Almacena el nombre completo del estudiante o usuario registrado.
email_usuario	Cadena (VARCHAR)	150 caracteres	Único (UNIQUE), No Nulo (NOT NULL)	Dirección de correo electrónico institucional o personal usada como credencial única de acceso.
fecha_nacimiento	Fecha (DATE)	(N/D)	Nulo permitido (N/D)	Fecha de nacimiento del usuario utilizada para verificación de perfil y métricas demográficas.

fecha_creacion	Fecha-Hora (TIMESTAMP)	(N/D)	Predeterminado (DEFAULT CURRENT_TIMESTAMP)	Marca de tiempo que registra el momento exacto en que la cuenta fue creada en el sistema.
5. JUSTIFICACIÓN TÉCNICA DE TIPOS DE DATOS Y RESTRICCIONES				

id_usuario (Entero / Auto-incremental):

- Justificación: Se seleccionó un tipo de dato entero de 8 Bytes para permitir la escala del sistema sin riesgo de desbordamiento de clave primaria (overflow). La propiedad auto-incremental delega la generación de IDs al motor PostgreSQL mediante secuencias, eliminando condiciones de carrera en peticiones concurrentes.

nombre_usuario y email_usuario (VARCHAR):

- Justificación: Se utiliza VARCHAR en lugar de CHAR para optimizar el almacenamiento dinámico en disco, evitando desperdiciar espacio con caracteres de relleno. Las longitudes máximas (100 y 150 respectivamente) previenen ataques de inyección y garantizan margen suficiente para nombres compuestos y correos institucionales extensos.
- Restricción UNIQUE: Garantiza la regla de negocio donde ningún usuario puede duplicar el correo electrónico dentro de la plataforma.

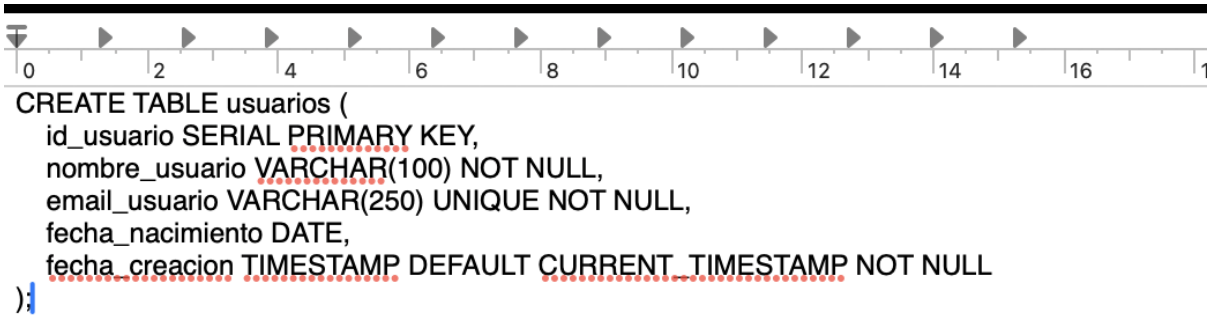
fecha_nacimiento (DATE) y fecha_creacion (TIMESTAMP):

- Justificación: Se separa el tipo fecha simple (DATE) para nacimientos, donde no se requiere precisión de hora, del tipo fecha-hora (TIMESTAMP)

para auditoría de registro, asegurando precisión milimétrica en los logs del sistema.

- Valor DEFAULT: La restricción de valor por defecto asigna automáticamente la fecha y hora del servidor al momento del registro, reduciendo la carga de procesamiento en la capa de aplicación.

6. SCRIPT SQL DE IMPLEMENTACIÓN EN POSTGRESQL El siguiente bloque de código corresponde al Script SQL DDL (Data Definition Language) ejecutado para la creación de la tabla en el entorno PostgreSQL:



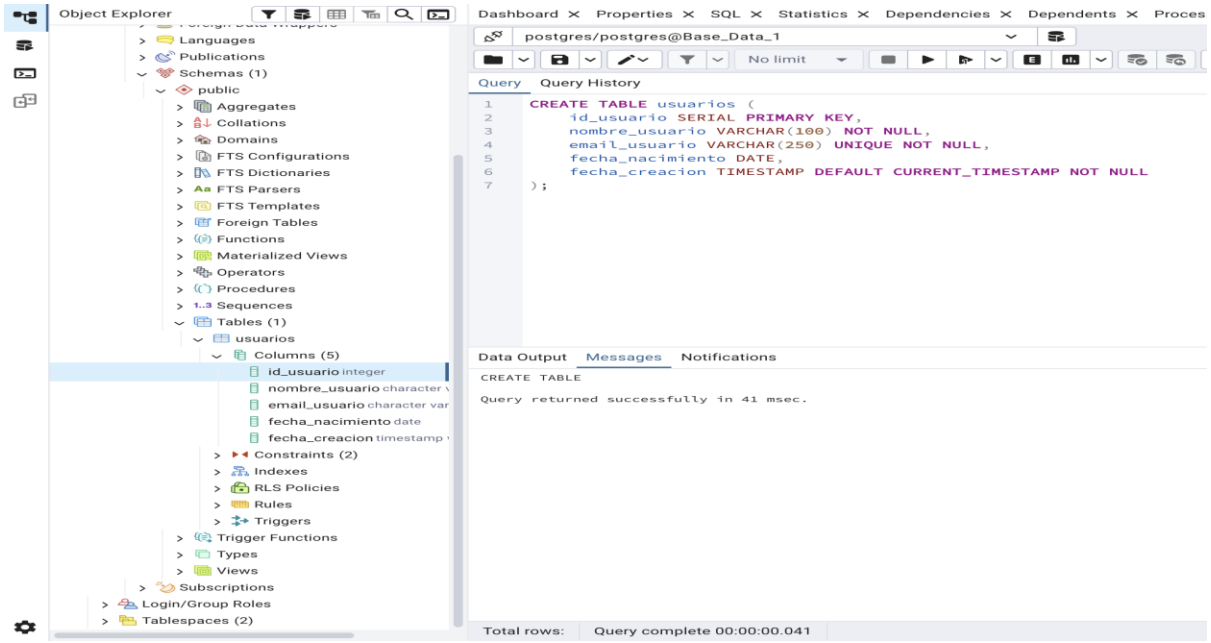
-- Creación de la tabla de usuarios para la Red Social Estudiantil CREATE TABLE usuarios (id_usuario SERIAL PRIMARY KEY, nombre_usuario VARCHAR(100) NOT NULL, email_usuario VARCHAR(150) UNIQUE NOT NULL, fecha_nacimiento DATE NULL, fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP)

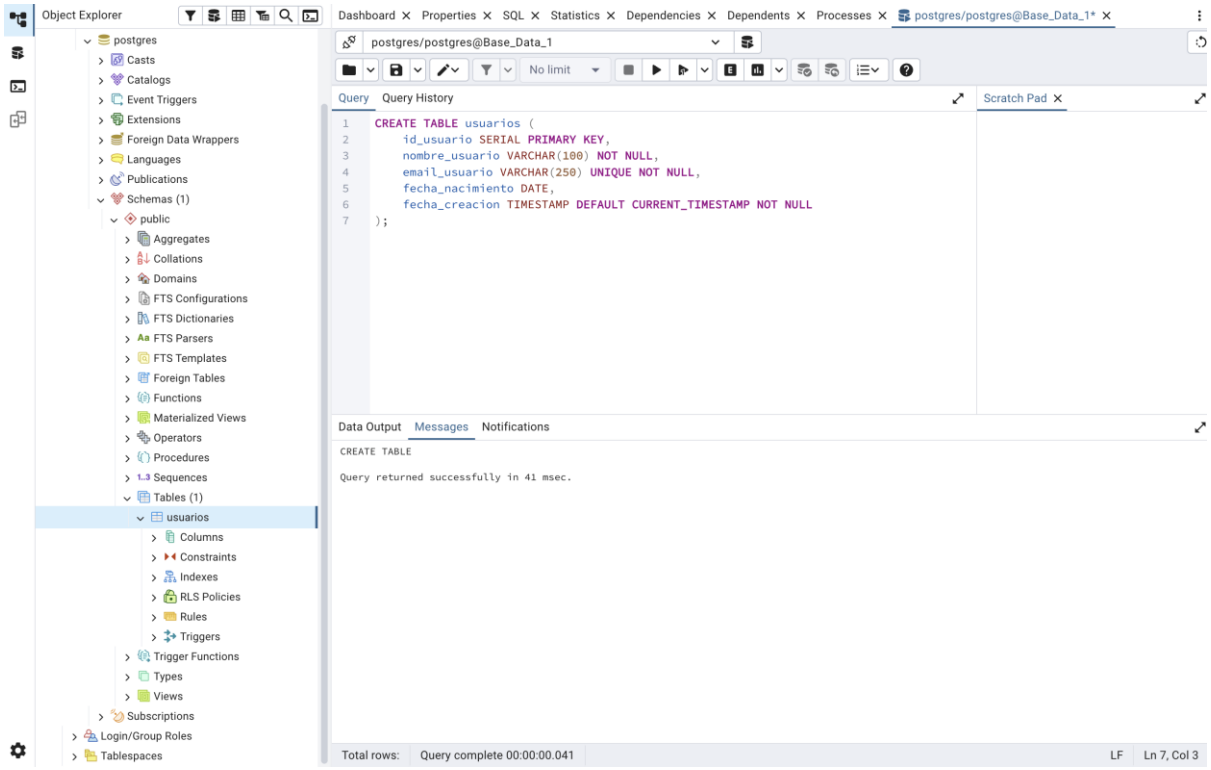
;

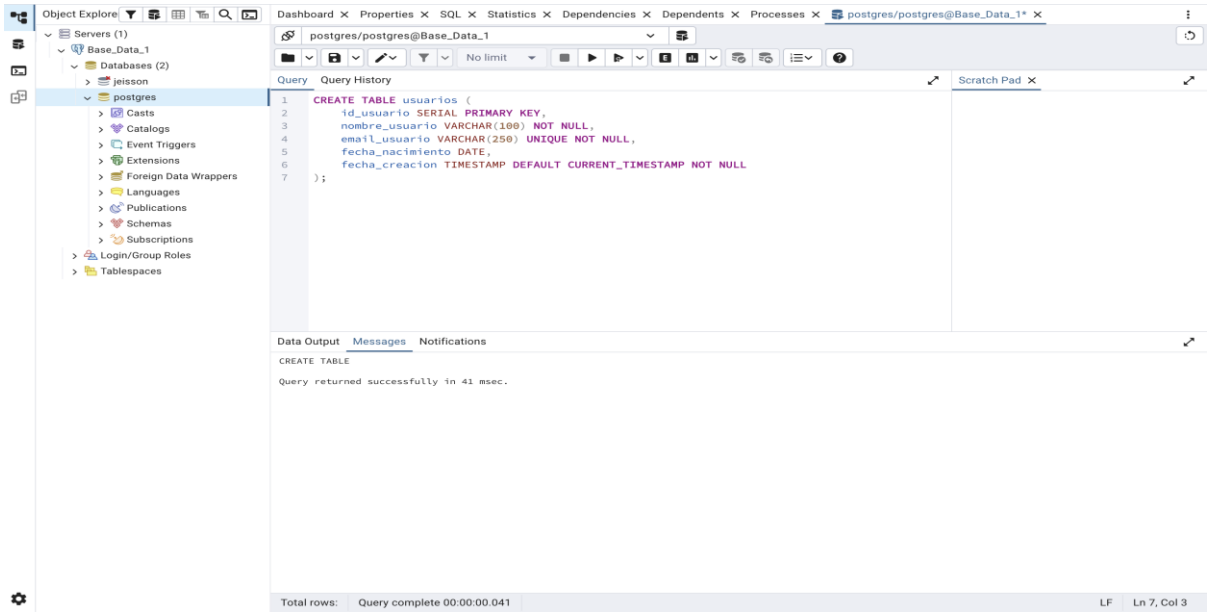
-- Comentarios explicativos sobre la estructura de la tabla COMMENT ON TABLE usuarios IS 'Tabla principal de registro de usuarios de la red social estudiantil.'; COMMENT ON COLUMN usuarios.id_usuario IS 'Clave primaria autoincrementable.'; COMMENT ON COLUMN usuarios.email_usuario IS 'Correo del usuario con restriccion de unicidad.';

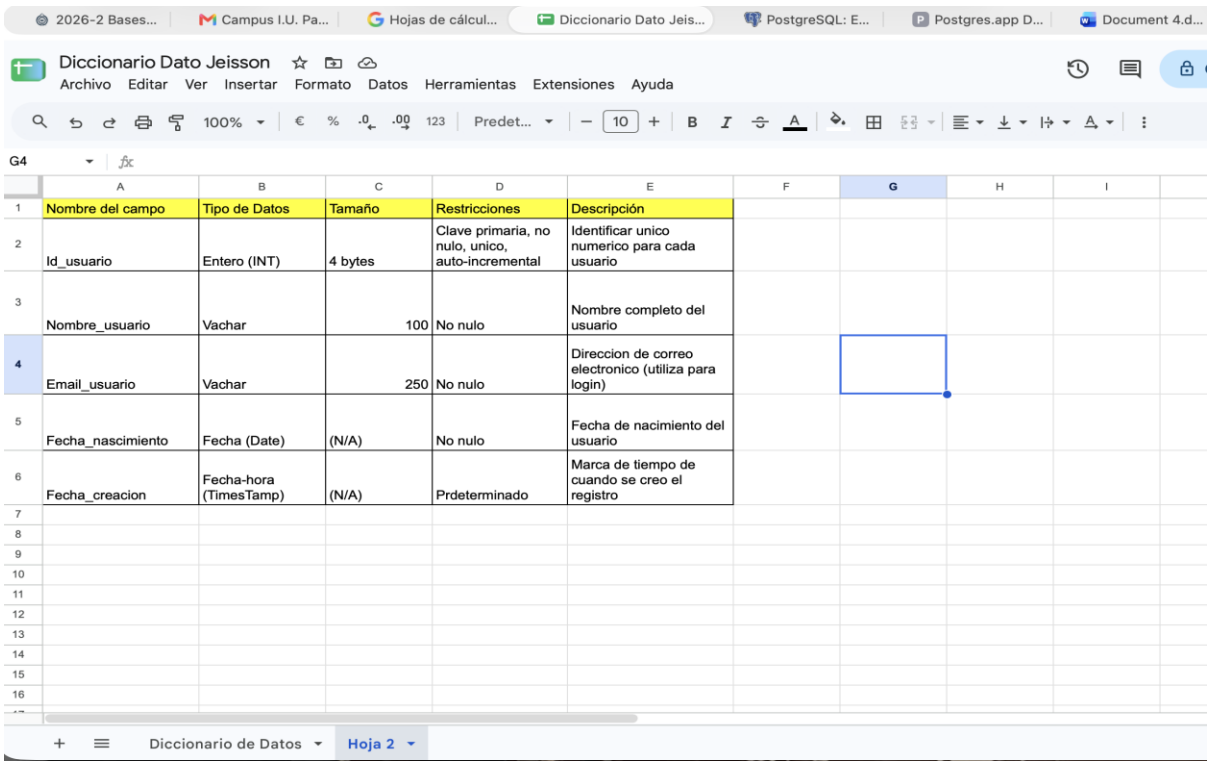
7. CONCLUSIONES

- La correcta definición de un diccionario de datos físico garantiza la consistencia del modelo relacional, sirviendo como contrato técnico entre los desarrolladores de software y los administradores de bases de datos (DBA).
- La elección fundamentada de tipos de datos (VARCHAR, INT, TIMESTAMP) optimiza el espacio de almacenamiento físico en disco y mejora los tiempos de respuesta en consultas e indexación.
- El uso de PostgreSQL como SGBD relacional ofrece un motor sólido, robusto y conforme al estándar ANSI SQL, permitiendo la portabilidad del script a otros motores como MySQL o Oracle con ajustes mínimos de sintaxis.

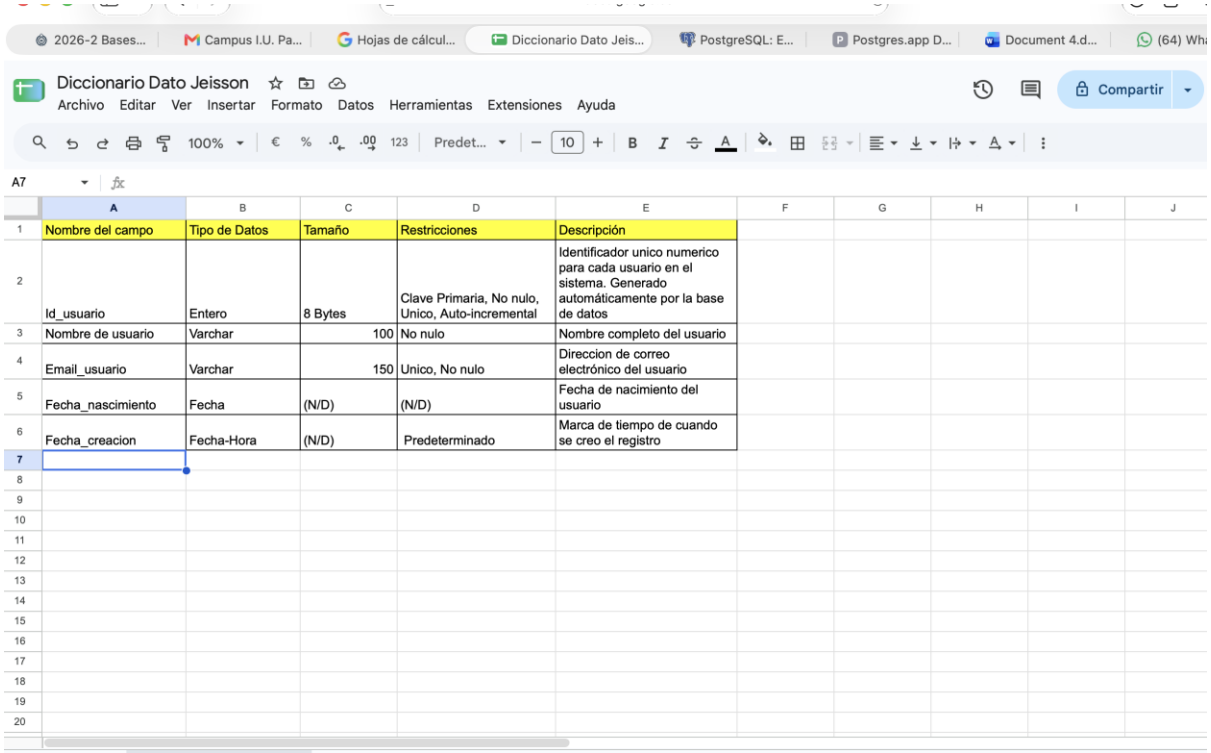

Object Explorer
Languages
Publications
Schemas (1)
public
Aggregates
Collations
Domains
FTS Configurations
FTS Dictionaries
FTS Parsers
FTS Templates
Foreign Tables
Functions
Materialized Views
Operators
Procedures
Sequences
Tables (1)
usuarios
Columns (5)
id_usuario integer
nombre_usuario character
email_usuario character var
fecha_nacimiento date
fecha_creacion timestamp
Constraints (2)
Indexes
RLS Policies
Rules
Triggers
Trigger Functions
Types
Views
Subscriptions
Login/Group Roles
Tablespaces (2)
Dashboard
Properties
SQL
Statistics
Dependencies
Dependents
Proces
postgres/postgres@Base_Data_1
No limit
Query
Query History
CREATE TABLE usuarios (
id_usuario SERIAL PRIMARY KEY,
nombre_usuario VARCHAR(100) NOT NULL,
email_usuario VARCHAR(250) UNIQUE NOT NULL,
fecha_nacimiento DATE,
fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL
);
Data Output
Messages
Notifications
CREATE TABLE
Query returned successfully in 41 msec.
Total rows:
Query complete 00:00:00.041


Object Explorer
postgres
Casts
Catalogs
Event Triggers
Extensions
Foreign Data Wrappers
Languages
Publications
Schemas (1)
public
Aggregates
Collations
Domains
FTS Configurations
FTS Dictionaries
FTS Parsers
FTS Templates
Foreign Tables
Functions
Materialized Views
Operators
Procedures
Sequences
Tables (1)
usuarios
Columns
Constraints
Indexes
RLS Policies
Rules
Triggers
Trigger Functions
Types
Views
Subscriptions
Login/Group Roles
Tablespaces
Dashboard
Properties
SQL
Statistics
Dependencies
Dependents
Processes
postgres/postgres@Base_Data_1*
postgres/postgres@Base_Data_1
No limit
Query
Query History
Scratch Pad
CREATE TABLE usuarios (
id_usuario SERIAL PRIMARY KEY,
nombre_usuario VARCHAR(100) NOT NULL,
email_usuario VARCHAR(250) UNIQUE NOT NULL,
fecha_nacimiento DATE,
fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL
);
Data Output
Messages
Notifications
CREATE TABLE
Query returned successfully in 41 msec.
Total rows:
Query complete 00:00:00.041
LF
Ln 7, Col 3





	A	B	C	D	E	F	G	H	I
1	Nombre del campo	Tipo de Datos	Tamaño	Restricciones	Descripción				
2	Id_usuario	Entero (INT)	4 bytes	Clave primaria, no nulo, unico, auto-incremental	Identificar unico numerico para cada usuario				
3	Nombre_usuario	Vachar	100	No nulo	Nombre completo del usuario				
4	Email_usuario	Vachar	250	No nulo	Direccion de correo electronico (utiliza para login)				
5	Fecha_nascimiento	Fecha (Date)	(N/A)	No nulo	Fecha de nacimiento del usuario				
6	Fecha_creacion	Fecha-hora (TimesTamp)	(N/A)	Prdeterminado	Marca de tiempo de cuando se creo el registro				
7									
8									
9									
10									
11									
12									
13									
14									
15									
16									



	A	B	C	D	E	F	G	H	I	J
1	Nombre del campo	Tipo de Datos	Tamaño	Restricciones	Descripción					
2	Id_usuario	Entero	8 Bytes	Clave Primaria, No nulo, Unico, Auto-incremental	Identificador unico numerico para cada usuario en el sistema. Generado automáticamente por la base de datos					
3	Nombre de usuario	Varchar	100	No nulo	Nombre completo del usuario					
4	Email_usuario	Varchar	150	Unico, No nulo	Direccion de correo electrónico del usuario					
5	Fecha_nascimiento	Fecha	(N/D)	(N/D)	Fecha de nacimiento del usuario					
6	Fecha_creacion	Fecha-Hora	(N/D)	Predeterminado	Marca de tiempo de cuando se creo el registro					
7										
8										
9										
10										
11										
12										
13										
14										
15										
16										
17										
18										
19										
20										